

МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА – Российский технологический университет»
РТУ МИРЭА

Институт информационных технологий (ИИТ)
Кафедра прикладной математики

ОТЧЕТ ПО ПРАКТИЧЕСКОЙ РАБОТЕ

по дисциплине «Технологии и инструментарий анализа больших данных»

Практическая работа №5
Машинное обучение в анализе данных
с использованием Spark MLlib

Студент группы ИНБО-12-23, Албахтин И.В. _____
(подпись)

Преподаватель Приходько Н.А. _____
(подпись)

Москва 2026 г.

Содержание

Практическая работа 5 3

1 Данные и контроль качества 4

2 Разбиение и исследование train 5

3 Полные пайплайны подготовки 6

4 Регрессия расхода энергии 7

5 Классификация тренировок 9

6 Кластеризация без целевой переменной 11

7 Проекция и интерпретация групп 12

8 Сохранение моделей и проверка загрузки 13

9 Результаты и ограничения 14

Источники 15

Практическая работа 5

Машинное обучение в PySpark на собственном наборе Calories

ФИО: Албахтин И.В.

Группа: ИНБО-12-23.

Дата выполнения: 08.11.2026.

Один предоставленный набор, 15 000 тренировок. Решаются регрессия Calories, классификация Calories >= 200 и кластеризация тренировок. Демонстрационные Online Shoppers и Seoul Bike в сдаваемую работу не входят. Порог 200 ккал фиксирован заранее и не является физиологической нормой. Момент прогноза - окончание тренировки, когда длительность, пульс и температура известны. User_ID и Calories исключены из признаков.

Для повторения положить исходный calories.csv рядом с блокнотом или в data/. SHA256 проверяется до расчётов. Нужны PySpark 3.5.7, JDK 17 и зависимости requirements-ml.txt.

In [1]:

```
from pathlib import Path
from datetime import datetime,timezone
from time import perf_counter
import hashlib,json,os,platform,subprocess
os.environ['MPLBACKEND']='Agg'
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from IPython.display import display,Image as NotebookImage
from sklearn.metrics import precision_recall_curve
from pyspark.sql import SparkSession,functions as F,types as T,Window
from pyspark.ml import Pipeline,PipelineModel
from pyspark.ml.feature import
StringIndexer,OneHotEncoder,VectorAssembler,StandardScaler,PCA
from pyspark.ml.regression import LinearRegression,RandomForestRegressor
from pyspark.ml.classification import LogisticRegression,RandomForestClassifier
from pyspark.ml.clustering import KMeans
from pyspark.ml.evaluation import
RegressionEvaluator,BinaryClassificationEvaluator,ClusteringEvaluator
from pyspark.ml.functions import vector_to_array
ROOT=Path.cwd();OUT=ROOT/'results/pr5';MODELS=ROOT/'models/pr5'
OUT.mkdir(parents=True,exist_ok=True);MODELS.mkdir(parents=True,exist_ok=True)
SEED=42;CLASS_THRESHOLD=200.0
spark=
(SparkSession.builder.master('local[*]').appName('TIABD_PR5_Calories_Albakhtin_IV'
)

.config('spark.driver.memory','1536m').config('spark.sql.shuffle.partitions','4')
       .config('spark.sql.session.timeZone','UTC').getOrCreate())
spark.sparkContext.setLogLevel('ERROR')
```

```
environment=
{'python':platform.python_version(),'spark':spark.version,'master':spark.sparkCont
ext.master,
             'parallelism':spark.sparkContext.defaultParallelism,
             'java':subprocess.run(['java','-
version'],capture_output=True,text=True).stderr.splitlines()[0]}
print(json.dumps(environment,ensure_ascii=False,indent=2))
```

```
26/11/08 16:38:14 WARN Utils: Your hostname, tiabd-albakhtin-iv resolves to a
loopback address: 127.0.1.1; using 10.0.2.15 instead (on interface enp0s3)
26/11/08 16:38:14 WARN Utils: Set SPARK_LOCAL_IP if you need to bind to another
address
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use
setLogLevel(newLevel).

26/11/08 16:38:15 WARN NativeCodeLoader: Unable to load native-hadoop library for
your platform... using builtin-java classes where applicable

{
  "python": "3.10.12",
  "spark": "3.5.7",
  "master": "local[*]",
  "parallelism": 2,
  "java": "openjdk version \"17.0.20.1\" 2026-08-18"
}
```

1 Данные и контроль качества

Calories выбран из предоставленных данных. Явная схема отличает ключ, категорию и числовые поля. Проверяются исходный хэш, 15 000 уникальных ключей, пропуски, дубликаты признаков и диапазоны. Цель не заполняется, строки не удаляются молча.

In [2]:

```
SOURCE=next((p for p in [ROOT/'data/calories.csv',ROOT/'calories.csv'] if
p.is_file()),None)
assert SOURCE is not None,'Положите исходный calories.csv в data/ или рядом с
блокнотом'
source_sha=hashlib.sha256(SOURCE.read_bytes()).hexdigest()
assert
source_sha=='2fa1f6ab63d1b835f1f849ee0825043e456fde84c177600577a7090916b9b7c4'
NUM=['Age','Height','Weight','Duration','Heart_Rate','Body_Temp']
schema=T.StructType([T.StructField('User_ID',T.LongType()),T.StructField('Gender',
T.StringType())]+
                    [T.StructField(c,T.DoubleType()) for c in NUM+['Calories']])
raw=spark.read.option('header',True).option('mode','FAILFAST').schema(schema).csv(
str(SOURCE)).cache()
nulls=raw.select([F.sum(F.col(c).isNull().cast('int')).alias(c) for c in
raw.columns]).first().asDict()
assert raw.count()==raw.select('User_ID').distinct().count()==15000
assert not any(nulls.values())
assert {r[0] for r in raw.select('Gender').distinct().collect()}==
{'male','female'}
assert not raw.filter('Age <= 0 OR Height <= 0 OR Weight <= 0 OR Duration <= 0 OR
Heart_Rate <= 0 OR Calories < 0').count()
duplicate_features=raw.count()-raw.select(*NUM,'Gender').distinct().count()
```

```
quality=
{'rows':15000,'columns':raw.columns,'schema':raw.schema.jsonValue(),'nulls':nulls,

'unique_ids':15000,'duplicate_feature_rows':duplicate_features,'source_sha256':sou
rce_sha}
raw.printSchema();raw.select(*NUM,'Calories').summary('min','max').show(truncate=F
alse)
print('NULL:',nulls,'Duplicate feature rows:',duplicate_features)
```

```
root
 |-- User_ID: long (nullable = true)
 |-- Gender: string (nullable = true)
 |-- Age: double (nullable = true)
 |-- Height: double (nullable = true)
 |-- Weight: double (nullable = true)
 |-- Duration: double (nullable = true)
 |-- Heart_Rate: double (nullable = true)
 |-- Body_Temp: double (nullable = true)
 |-- Calories: double (nullable = true)


+-------+----+------+------+--------+----------+---------+--------+
|summary|Age |Height|Weight|Duration|Heart_Rate|Body_Temp|Calories|
+-------+----+------+------+--------+----------+---------+--------+
|min    |20.0|123.0 |36.0  |1.0     |67.0      |37.1     |1.0     |
|max    |79.0|222.0 |132.0 |30.0    |128.0     |41.5     |314.0   |
+-------+----+------+------+--------+----------+---------+--------+

NULL: {'User_ID': 0, 'Gender': 0, 'Age': 0, 'Height': 0, 'Weight': 0, 'Duration':
0, 'Heart_Rate': 0, 'Body_Temp': 0, 'Calories': 0} Duplicate feature rows: 1
```

2 Разбиение и исследование train

Для двух прогнозных задач используются общие непересекающиеся части. Строки сортируются по SHA256(User_ID:42): первые 9 000 - train, следующие 3 000 - valid, остальные 3 000 - test. Модели и порог выбираются на valid; test открывается после выбора. Распределения изучаются только на train. Кластеризация далее - отдельный описательный анализ всех тренировок.

In [3]:

```
window=Window.orderBy(F.sha2(F.concat_ws(':',F.col('User_ID').cast('string'),F.lit
(str(SEED))),256),'User_ID')
data=(raw.withColumn('label',F.col('Calories')).withColumn('class_label',
(F.col('Calories')>=CLASS_THRESHOLD).cast('double'))

.withColumn('_rank',F.row_number().over(window)).withColumn('split',F.when(F.col('
_rank')<=9000,'train')
      .when(F.col('_rank')
<=12000,'valid').otherwise('test')).drop('_rank').cache())
parts={name:data.filter(F.col('split')==name) for name in
['train','valid','test']}
train,valid,test=[parts[name] for name in ['train','valid','test']]
assert [p.count() for p in parts.values()]==[9000,3000,3000]
assert
all(parts[a].select('User_ID').intersect(parts[b].select('User_ID')).count()==0
          for a,b in [('train','valid'),('train','test'),('valid','test')])
```

```
def local(df):
    frame=df.orderBy('User_ID').toPandas()
    frame['row_key']=frame.pop('User_ID').map(lambda
x:hashlib.sha256(str(int(x)).encode()).hexdigest()[:16])
    return frame
local(data.select('User_ID','split','class_label')).to_csv(OUT/'split.csv',index=F
alse)
split_counts={name:part.count() for name,part in parts.items()}
class_balance=[r.asDict() for r in
data.groupBy('split','class_label').count().orderBy('split','class_label').collect
()]
print('Split:',split_counts);print(pd.DataFrame(class_balance).to_string(index=Fal
se))
eda=train.select(*NUM,'Gender','Calories').toPandas()
eda.describe(include='all').to_csv(OUT/'train_distribution.csv')
fig,axes=plt.subplots(2,4,figsize=(13,6))
for ax,c in zip(axes.flat,NUM+['Calories','Gender']):
    if c=='Gender':eda[c].value_counts().plot.bar(ax=ax)
    else:ax.hist(eda[c],bins=30)
    ax.set(title=c,ylabel='Train rows')
fig.tight_layout();fig.savefig(OUT/'train_distribution.png',dpi=160);plt.show();pl
t.close(fig)
display(NotebookImage(filename=str(OUT/'train_distribution.png')))

Split: {'train': 9000, 'valid': 3000, 'test': 3000}
split  class_label  count
 test          0.0   2833
 test          1.0    167
train          0.0   8555
train          1.0    445
valid          0.0   2830
valid          1.0    170
```

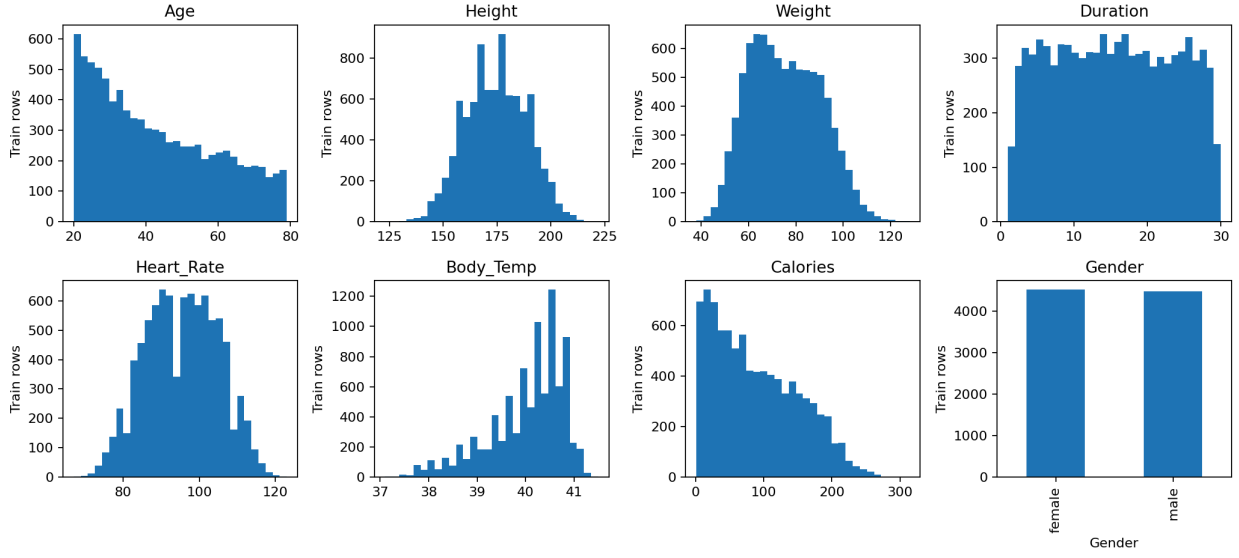


Рисунок 1 – Распределения предикторов и цели на train

3 Полные пайплайны подготовки

StringIndexer, OneHotEncoder и StandardScaler обучаются внутри Pipeline на train. Gender кодируется без предположения о порядке; одна из двух категорий опускается, чтобы избежать линейной зависимости с intercept. Домен

male/female проверяется до обучения, неизвестные значения вызывают ошибку контракта. Масштабирование без центрирования сохраняет разреженность. Обе цели, ключ и split исключены из признаков.

In [4]:

```
def supervised_pipeline(estimator):
    return Pipeline(stages=
[StringIndexer(inputCol='Gender',outputCol='gender_idx',handleInvalid='error'),

OneHotEncoder(inputCol='gender_idx',outputCol='gender_ohe',handleInvalid='error',d
ropLast=True),
        VectorAssembler(inputCols=NUM+['gender_ohe'],outputCol='features_raw'),

StandardScaler(inputCol='features_raw',outputCol='features',withMean=False,withStd
=True),estimator])
def reg_metrics(pred):
    return
{name:RegressionEvaluator(labelCol='label',predictionCol='prediction',metricName=n
ame).evaluate(pred)
            for name in ['rmse','mae','r2']}
def confusion(y,score,threshold):
    y=np.asarray(y,dtype=int);pred=np.asarray(score)>=threshold
    tp=int(np.sum((y==1)&pred));tn=int(np.sum((y==0)&~pred))
    fp=int(np.sum((y==0)&pred));fn=int(np.sum((y==1)&~pred))
    precision=tp/(tp+fp) if tp+fp else 0.;recall=tp/(tp+fn) if tp+fn else 0.
    return {'tp':tp,'tn':tn,'fp':fp,'fn':fn,'accuracy':
(tp+tn)/len(y),'precision':precision,
            'recall':recall,'f1':2*precision*recall/(precision+recall) if
precision+recall else 0.}
def scores(model,frame):
    return
model.transform(frame).select('User_ID','class_label',vector_to_array('probability
')[1].alias('score'))
def cls_metrics(frame,threshold=.5):
    p=frame.select('class_label','score').toPandas()
    result=confusion(p.class_label,p.score,threshold)
    for metric in ['areaUnderPR','areaUnderROC']:

result[metric]=BinaryClassificationEvaluator(labelCol='class_label',rawPredictionC
ol='score',metricName=metric,numBins=0).evaluate(frame)
    return result
print('Predictors:',NUM+['Gender'],'; excluded: User_ID, Calories, label,
class_label, split')

Predictors: ['Age', 'Height', 'Weight', 'Duration', 'Heart_Rate', 'Body_Temp',
'Gender'] ; excluded: User_ID, Calories, label, class_label, split
```

4 Регрессия расхода энергии

Сравниваются LinearRegression с regParam 0,01 и 0,1 и RandomForestRegressor из 80 деревьев с глубиной 8 и 12. Выбор - минимум RMSE valid. Базовое решение - среднее Calories train. После выбора модель оценивается на test. RMSE и MAE измеряются в ккал. Показаны прогноз и остатки той же test.

In [5]:

```
reg_rows=[];reg_models={}
for name,est in
[('LR_0.01',LinearRegression(regParam=.01,solver='normal',standardization=False)),

('LR_0.1',LinearRegression(regParam=.1,solver='normal',standardization=False)),

('RF_8',RandomForestRegressor(numTrees=80,maxDepth=8,minInstancesPerNode=3,seed=SE
ED)),

('RF_12',RandomForestRegressor(numTrees=80,maxDepth=12,minInstancesPerNode=3,seed=
SEED))]:
    start=perf_counter();model=supervised_pipeline(est).fit(train)
    reg_models[name]=model;reg_rows.append({'model':name,'fit_s':perf_counter()-
start,**reg_metrics(model.transform(valid))})
reg_best=min(reg_rows,key=lambda r:r['rmse'])
['model'];reg_model=reg_models[reg_best]
train_mean=train.agg(F.avg('Calories')).first()[0]
reg_test=reg_model.transform(test).select('User_ID','label','prediction').cache()
reg_test_metrics=reg_metrics(reg_test)
reg_baseline=reg_metrics(test.withColumn('prediction',F.lit(train_mean)))
reg_frame=local(reg_test);reg_frame['baseline']=train_mean
reg_frame.to_csv(OUT/'regression_test_predictions.csv',index=False)
pd.DataFrame(reg_rows).to_csv(OUT/'regression_validation.csv',index=False)
print(pd.DataFrame(reg_rows).to_string(index=False));print('Selected:',reg_best,'T
est:',reg_test_metrics,'Baseline:',reg_baseline)
fig,axes=plt.subplots(1,2,figsize=(10,4))
axes[0].scatter(reg_frame.label,reg_frame.prediction,s=8,alpha=.3)
limits=
[min(reg_frame.label.min(),reg_frame.prediction.min()),max(reg_frame.label.max(),r
eg_frame.prediction.max())]
axes[0].plot(limits,limits,'r--');axes[0].set(xlabel='Факт, ккал',ylabel='Прогноз,
ккал',title='Регрессия Calories: test')
axes[1].hist(reg_frame.label-
reg_frame.prediction,bins=40);axes[1].set(xlabel='Ошибка, ккал',title='Остатки
test')
fig.tight_layout();fig.savefig(OUT/'regression_test.png',dpi=160);plt.show();plt.c
lose(fig)
display(NotebookImage(filename=str(OUT/'regression_test.png')))
```

```
[Stage 172:>                                                        (0 + 1) / 1]
```

```
[Stage 175:>                                                        (0 + 1) / 1]
```

```
[Stage 176:>                                                        (0 + 1) / 1]
```

```
[Stage 178:>                                                        (0 + 1) / 1]
```

```
[Stage 179:>                                                        (0 + 1) / 1]
```

```
  model    fit_s      rmse      mae       r2
LR_0.01 1.111137 11.498791 8.415733 0.966245
 LR_0.1 0.389188 11.495845 8.409047 0.966262
   RF_8 2.672626  5.784483 3.746630 0.991458
```

```
  RF_12 9.491686  4.103530 2.455335 0.995701
Selected: RF_12 Test: {'rmse': 4.114613853306854, 'mae': 2.3875149703757255, 'r2':
0.9957679977330948} Baseline: {'rmse': 63.25909577061658, 'mae':
53.65172333333293, 'r2': -0.0003078646128016338}
```

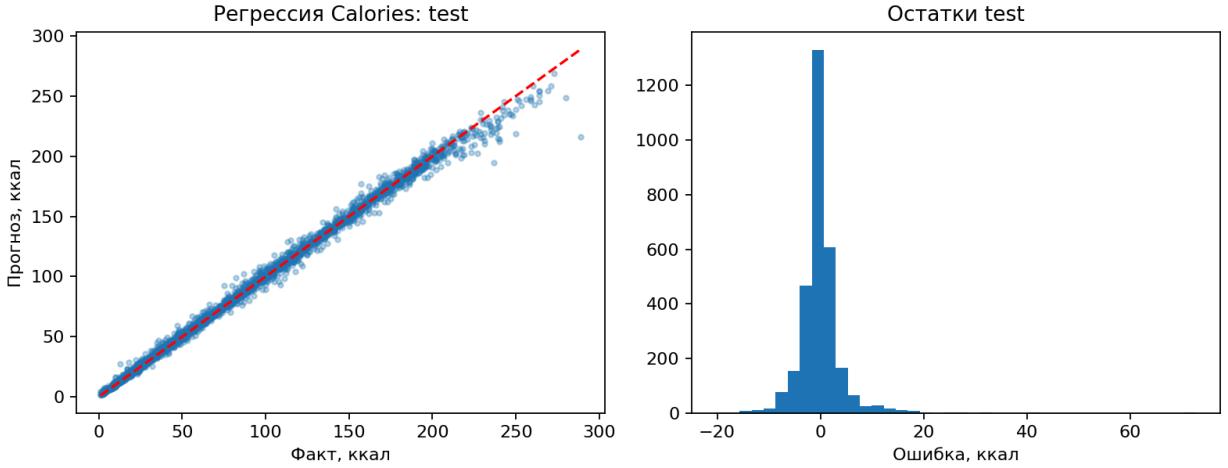


Рисунок 2 – Прогноз Calories и остатки на test

5 Классификация тренировок

Цель - Calories >= 200. LogisticRegression с regParam 0,01 и 0,1 и RandomForestClassifier из 80 деревьев с глубиной 6 и 10 сравниваются по PR-AUC valid. Для выбранной модели порог вероятности выбирается по F1 valid из сетки 0,10…0,90. Фиксированный порог цели 200 ккал отличается от подбираемого порога вероятности. Базовое решение выдаёт наиболее частый класс train.

In [6]:

```
cls_rows=[];cls_models={};cls_valid_frames={}
for name,est in
[('LR_0.01',LogisticRegression(labelCol='class_label',regParam=.01,maxIter=150,sta
ndardization=False)),

('LR_0.1',LogisticRegression(labelCol='class_label',regParam=.1,maxIter=150,standa
rdization=False)),

('RF_6',RandomForestClassifier(labelCol='class_label',numTrees=80,maxDepth=6,minIn
stancesPerNode=3,seed=SEED)),

('RF_10',RandomForestClassifier(labelCol='class_label',numTrees=80,maxDepth=10,min
InstancesPerNode=3,seed=SEED))]:
    start=perf_counter();model=supervised_pipeline(est).fit(train)
    cls_models[name]=model;p=scores(model,valid).cache()
    cls_rows.append({'model':name,'fit_s':perf_counter()-start,**cls_metrics(p)})
    cls_valid_frames[name]=local(p);p.unpersist()
cls_best=max(cls_rows,key=lambda r:r['areaUnderPR'])
['model'];cls_model=cls_models[cls_best]
v=cls_valid_frames[cls_best]
threshold_grid=[{'threshold':float(t),**confusion(v.class_label,v.score,t)} for t
in np.linspace(.1,.9,17)]
prob_threshold=min(threshold_grid,key=lambda r:(-
```

```
r['f1'],abs(r['threshold']-.5),r['threshold']))['threshold']
cls_test=scores(cls_model,test).cache();cls_test_metrics=cls_metrics(cls_test,prob
_threshold)
cls_frame=local(cls_test);cls_frame['prediction']=
(cls_frame.score>=prob_threshold).astype(int)
majority=int(train.agg(F.avg('class_label')).first()[0]>=.5)
cls_baseline=confusion(cls_frame.class_label,np.full(len(cls_frame),majority),.5)
cls_frame.to_csv(OUT/'classification_test_predictions.csv',index=False)
pd.DataFrame(cls_rows).to_csv(OUT/'classification_validation.csv',index=False)
pd.DataFrame(threshold_grid).to_csv(OUT/'classification_thresholds.csv',index=Fals
e)
pd.concat([frame.assign(model=name) for name,frame in
cls_valid_frames.items()]).to_csv(OUT/'classification_valid_scores.csv',index=Fals
e)
print(pd.DataFrame(cls_rows)
[['model','areaUnderPR','areaUnderROC','f1']].to_string(index=False))
print('Selected:',cls_best,'Threshold:',prob_threshold,'Test:',cls_test_metrics,'M
ajority baseline:',cls_baseline)
fig,axes=plt.subplots(1,2,figsize=(10,4))
for name,p in cls_valid_frames.items():

precision,recall,_=precision_recall_curve(p.class_label,p.score);axes[0].plot(reca
ll,precision,label=name)
axes[0].set(xlabel='Recall',ylabel='Precision',title='PR-кривые
valid');axes[0].legend()
m=cls_test_metrics;cm=np.array([[m['tn'],m['fp']],[m['fn'],m['tp']]])
axes[1].imshow(cm,cmap='Blues')
for i in range(2):
    for j in range(2):axes[1].text(j,i,str(cm[i,j]),ha='center',va='center')
axes[1].set(xticks=[0,1],yticks=
[0,1],xlabel='Прогноз',ylabel='Факт',title='Матрица ошибок test')
fig.tight_layout();fig.savefig(OUT/'classification_test.png',dpi=160);plt.show();p
lt.close(fig)
display(NotebookImage(filename=str(OUT/'classification_test.png')))
```

```
  model  areaUnderPR  areaUnderROC       f1
LR_0.01     0.911916      0.992444 0.547009
 LR_0.1     0.852560      0.984569 0.000000
   RF_6     0.968463      0.997468 0.853333
  RF_10     0.973470      0.998140 0.889590
Selected: RF_10 Threshold: 0.35 Test: {'tp': 161, 'tn': 2816, 'fp': 17, 'fn': 6,
'accuracy': 0.9923333333333333, 'precision': 0.9044943820224719, 'recall':
0.9640718562874252, 'f1': 0.9333333333333333, 'areaUnderPR': 0.9791770936534788,
'areaUnderROC': 0.9988205727619945} Majority baseline: {'tp': 0, 'tn': 2833, 'fp':
0, 'fn': 167, 'accuracy': 0.9443333333333334, 'precision': 0.0, 'recall': 0.0,
'f1': 0.0}
```

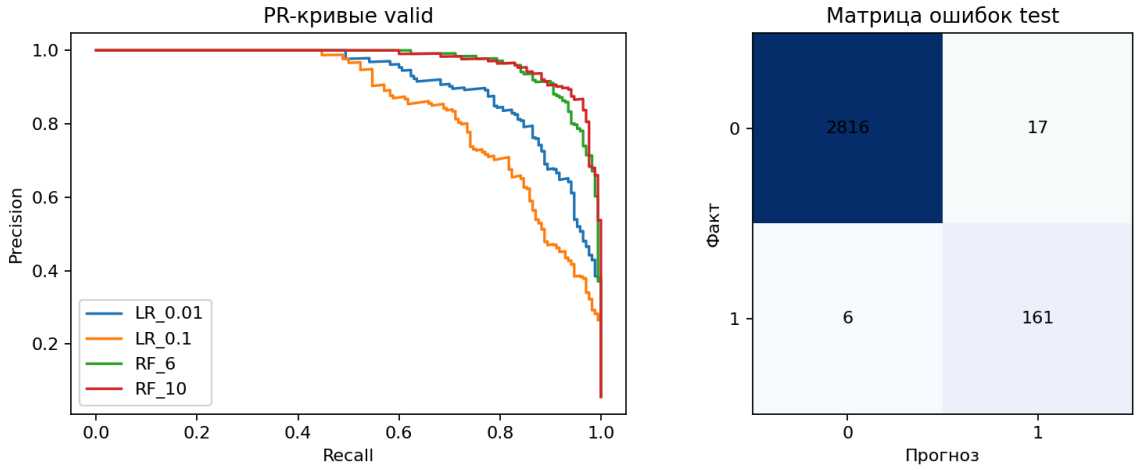


Рисунок 3 – PR-кривые valid и матрица ошибок test

6 Кластеризация без целевой переменной

KMeans обучается на всех 15 000 тренировках по шести числовым предикторам. Gender, ключ и Calories исключены. StandardScaler с центрированием и KMeans входят в Pipeline. Для k=2…8 силуэт squaredEuclidean вычисляется на одной фиксированной выборке 2 000 строк; выбирается максимум. Это геометрический критерий, а не точность прогнозирования Calories.

In [7]:

```
sample=raw.orderBy(F.sha2(F.concat_ws(':',F.col('User_ID').cast('string'),F.lit('c
luster42')),256),'User_ID').limit(2000).cache()
cluster_models={};k_rows=[];sample_frames=[]
for k in range(2,9):
    pipeline=Pipeline(stages=
[VectorAssembler(inputCols=NUM,outputCol='cluster_raw'),

StandardScaler(inputCol='cluster_raw',outputCol='features',withMean=True,withStd=T
rue),
        KMeans(k=k,seed=SEED,maxIter=50,initSteps=3,tol=1e-4)])
    start=perf_counter();model=pipeline.fit(raw);cluster_models[k]=model
    pred=model.transform(sample)

silhouette=ClusteringEvaluator(distanceMeasure='squaredEuclidean').evaluate(pred)
    counts=model.transform(raw).groupBy('prediction').count().collect()

k_rows.append({'k':k,'silhouette':silhouette,'training_cost':model.stages[-1].summ
ary.trainingCost,
                   'min_cluster':min(r['count'] for r in
counts),'fit_s':perf_counter()-start})

sample_frames.append(local(pred.select('User_ID',F.col('prediction').alias('cluste
r'))).assign(k=k))
best_k=max(k_rows,key=lambda r:r['silhouette'])
['k'];cluster_model=cluster_models[best_k]
clustered=cluster_model.transform(raw).cache()
profiles=[r.asDict() for r in
```

```
clustered.groupBy('prediction').agg(F.count('*').alias('n'),
            *[F.avg(c).alias(c) for c in NUM+
['Calories']]).orderBy('prediction').collect()]
pd.DataFrame(profiles).to_csv(OUT/'cluster_profiles.csv',index=False)
pd.DataFrame(k_rows).to_csv(OUT/'kmeans_selection.csv',index=False)
pd.concat(sample_frames).to_csv(OUT/'silhouette_assignments.csv',index=False)
print(pd.DataFrame(k_rows).to_string(index=False));print('Selected k:',best_k)
print(pd.DataFrame(profiles).round(2).to_string(index=False))
```

```
 k  silhouette  training_cost  min_cluster    fit_s
 2    0.464170   61182.450846         6552 1.180232
 3    0.426842   47710.174647         4767 1.160579
 4    0.392124   40628.782636         3265 0.844686
 5    0.394804   37202.443475         2060 1.038777
 6    0.373014   33224.076931         1953 1.007228
 7    0.374960   30324.670801         1582 0.967208
 8    0.366780   27901.661503         1573 0.796659
Selected k: 2
 prediction    n   Age  Height  Weight  Duration  Heart_Rate  Body_Temp  Calories
          0 8448 42.82  174.32   74.83     21.70      102.11      40.56    133.77
          1 6552 42.75  174.65   75.14      7.58       87.02      39.34     32.51
```

7 Проекция и интерпретация групп

PCA служит только изображению кластеров; силуэт рассчитан в исходном шестимерном пространстве. Средний Calories добавляется к профилям после обучения, не влияет на группы. Номера кластеров условны, а двумерная проекция сохраняет лишь часть вариации.

In [8]:

```
pca=PCA(k=2,inputCol='features',outputCol='pca').fit(clustered)
projected=pca.transform(clustered).select('User_ID',F.col('prediction').alias('clu
ster'),
            vector_to_array('pca')[0].alias('pc1'),vector_to_array('pca')
[1].alias('pc2'))
assignments=local(projected);assignments.to_csv(OUT/'cluster_assignments.csv',inde
x=False)
fig,axes=plt.subplots(1,2,figsize=(11,4))
axes[0].plot([r['k'] for r in k_rows],[r['silhouette'] for r in k_rows],'o-')
axes[0].set(xlabel='k',ylabel='Silhouette (squared Euclidean)',title='Одна выборка
2 000 строк')
for label,frame in assignments.groupby('cluster'):
    axes[1].scatter(frame.pc1,frame.pc2,s=4,alpha=.25,label=str(label))
axes[1].set(xlabel='PC1',ylabel='PC2',title=f'Все 15 000 тренировок, k=
{best_k}');axes[1].legend(title='Кластер')
fig.tight_layout();fig.savefig(OUT/'clustering.png',dpi=160);plt.show();plt.close(
fig)
display(NotebookImage(filename=str(OUT/'clustering.png')))
```

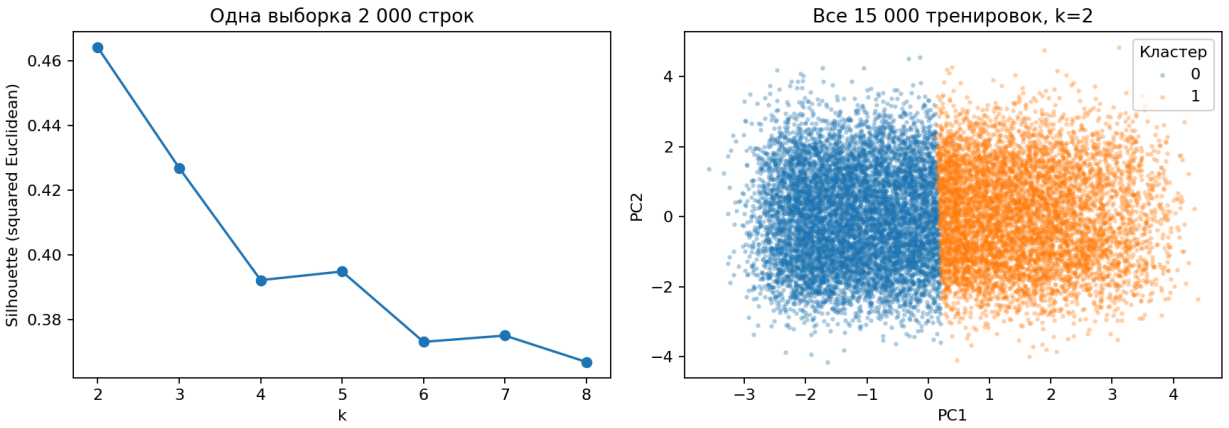


Рисунок 4 – Выбор k и PCA всех тренировок

8 Сохранение моделей и проверка загрузки

Три выбранных PipelineModel сохраняются с обученными словарями и масштабами. После чтения проверяются максимальные расхождения прогноза регрессии, вероятности классификации на test и номеров кластеров на полном наборе. Порог вероятности и параметры постановки сохраняются отдельно в JSON.

In [9]:

```
roundtrip={}
for name,model,frame,col in [('regression',reg_model,test,'prediction'),
                             ('classification',cls_model,test,'probability'),
                             ('clustering',cluster_model,raw,'prediction')]:
    path=MODELS/('calories_'+name);model.write().overwrite().save(str(path))

loaded=PipelineModel.load(str(path));left=model.transform(frame);right=loaded.tran
sform(frame)
    if name=='classification':
        left=left.select('User_ID',vector_to_array(col)[1].alias('before'))
        right=right.select('User_ID',vector_to_array(col)[1].alias('after'))
    else:

left=left.select('User_ID',F.col(col).alias('before'));right=right.select('User_ID
',F.col(col).alias('after'))
    error=left.join(right,'User_ID').agg(F.max(F.abs(F.col('before')-
F.col('after')))).first()[0]
    roundtrip[name]=float(error);assert error<1e-10
print('Max differences after PipelineModel.load:',roundtrip)
```

```
[Stage 1312:>                                                       (0 + 2) / 2]

[Stage 1312:============================>                         (1 + 1) / 2]


Max differences after PipelineModel.load: {'regression': 0.0, 'classification':
0.0, 'clustering': 0.0}
```

9 Результаты и ограничения

Все три задачи используют Calories. Прогнозы, параметры, профили и графики сохранены в results/pr5, модели - в models/pr5. Отдельный verify_pr5.py восстанавливает части по исходному CSV, пересчитывает метрики и семь силуэтов. Два ядра VM не измеряют масштабируемость кластера; один split не даёт универсального рейтинга моделей. Происхождение предоставленного Calories ограничено сведениями файла, поэтому клинические и причинные выводы не делаются.

In [10]:

```
summary={'submission_basis':'Moodle PR5: own dataset, regression, classification
and clustering; demonstration code not required',
 'environment':environment,'dataset':quality,'predictors':NUM+
['Gender'],'excluded_predictors':
['User_ID','Calories','label','class_label','split'],

'split':split_counts,'class_balance':class_balance,'seed':SEED,'class_label_thresh
old_kcal':CLASS_THRESHOLD,
 'regression':
{'validation':reg_rows,'selected_model':reg_best,'test':reg_test_metrics,'baseline
_test':reg_baseline,'train_mean':train_mean,

'scaler_std':list(reg_model.stages[3].std),'indexer_labels':reg_model.stages[0].la
bels},
 'classification':
{'validation':cls_rows,'selected_model':cls_best,'probability_threshold':prob_thre
shold,'test':cls_test_metrics,

'baseline_test':cls_baseline,'majority_train':majority,'threshold_grid':threshold_
grid},
 'clustering':
{'features':NUM,'fit_rows':15000,'silhouette_sample_rows':2000,'distance':'squared
Euclidean','k_selection':k_rows,

'selected_k':best_k,'profiles':profiles,'scaler_mean':list(cluster_model.stages[1]
.mean),

'scaler_std':list(cluster_model.stages[1].std),'pca_explained_variance':list(pca.e
xplainedVariance)},

'roundtrip_max_difference':roundtrip,'finished_utc':datetime.now(timezone.utc).iso
format()}
(OUT/'summary.json').write_text(json.dumps(summary,ensure_ascii=False,indent=2,all
ow_nan=False)+'\n',encoding='utf-8')
print(f'Регрессия: {reg_best}; RMSE test={reg_test_metrics["rmse"]:.4f}, baseline=
{reg_baseline["rmse"]:.4f} ккал.')
print(f'Классификация: {cls_best}; F1 test={cls_test_metrics["f1"]:.4f}, PR-AUC=
{cls_test_metrics["areaUnderPR"]:.4f}.')
print(f'Кластеризация: k={best_k}; silhouette={max(r["silhouette"] for r in
k_rows):.4f}; все 15 000 строк.')
for frame in [reg_test,cls_test,clustered,sample,data,raw]:frame.unpersist()
spark.stop();print('SparkSession stopped')
```

```
Регрессия: RF_12; RMSE test=4.1146, baseline=63.2591 ккал.
Классификация: RF_10; F1 test=0.9333, PR-AUC=0.9792.
Кластеризация: k=2; silhouette=0.4642; все 15 000 строк.

SparkSession stopped
```

Источники

1. Условия ПР5 в Moodle, предоставленные пользователем 08.11.2026: собственный набор, регрессия, классификация и кластеризация; сдача ipynb и PDF-экспорта.
2. Предоставленный calories.csv, 15 000 строк. SHA256: 2fa1f6ab63d1b835f1f849ee0825043e456fde84c177600577a7090916b9b7c4. Лицензия и способ измерения цели в исходных материалах не описаны.
3. Apache Spark 3.5.7 ML Pipelines.
4. Apache Spark 3.5.7 ClusteringEvaluator.